

Greedy random adaptive memory programming search for the capacitated clustering problem

Samad Ahmadi ^{a,*}, Ibrahim H. Osman ^b

^a School of Computing, De Montfort University, The Gateway, Leicester LE1 9BH, UK

^b Center for Advanced Mathematical Studies and School of Business, American University of Beirut, Beirut, Lebanon

Received 1 August 2001; accepted 14 August 2003

Available online 24 January 2004

Abstract

In the capacitated clustering problem (CCP), a given set of n weighted points is to be partitioned into p clusters such that, the total weight of the points in each cluster does not exceed a given cluster capacity. The objective is to find a set of p centers that minimises the total scatter of points allocated to these centers. In this paper, we propose a merger of Greedy Random Adaptive Search Procedure (GRASP) and Adaptive Memory Programming (AMP) into a new GRAMPS framework for the CCP. A learning process is kept in charge of tracking information on the best components in an elite set of GRAMPS solutions. The information are strategically combined with problem-domain data to restart the construction search phase. At early stage of constructions, priorities are given to problem-domain data and progressively shifted towards generated information as the learning increases. GRAMPS is implemented with an efficient local search descent based on a restricted λ -interchange neighbourhood. Extensive experiments are reported on a standard set of bench-marks from the literature and on a new set of large instances. The results show that GRAMPS has an efficient learning mechanism and is competitive with the existing methods in the literature.

© 2003 Elsevier B.V. All rights reserved.

Keywords: Ant colony optimization; Adaptive memory programming; Density search; Capacitated clustering (p -median) problem; Greedy randomized adaptive search procedure; Guided construction search metaheuristic

1. Introduction

In the capacitated clustering problem (CCP), a given set of n weighted points is to be partitioned into p clusters such that, the total weight of points in each cluster does not exceed a given cluster

capacity. The objective is to find a set of p centers—one center for each cluster—and to assign each point to exactly one cluster such that the total scatter Z of assigned points to their corresponding centers is minimized. The CCP is a combinatorial optimization of the following form. Given a set $A = \{a_1, \dots, a_n\}$ of n points, an integer number $p \geq 2$ and a cost (distance) matrix $C = (C_{ik})_{n \times n}$ where $C_{ik} = C(a_i, a_k)$ is the cost of assigning point a_i to center $a_k \in A$ with $C_{ik} \geq 0$. Each point a_i is associated with a positive demand d_i . Each cluster

* Corresponding author. Tel.: +44-116-2506314; fax: +44-116-2577936.

E-mail addresses: sahmadi@dmu.ac.uk (S. Ahmadi), ibrahim.osman@aub.edu.lb (I.H. Osman).

S_k contains a non-empty subset of points in A with a given cluster capacity Q . Let $S = \{S_1, \dots, S_p\}$ be the set of p clusters, $\Phi = \{\varphi_1, \dots, \varphi_p\}$ be the corresponding set of centers and $K = \{1, \dots, p\}$ be the set of indices for centers.

$$\begin{aligned} \text{The total scatter: } Z &= \sum_{a_i \in S_k} \sum_{k \in K} C_{(a_i, \varphi_k)} \\ \text{subject to: } A &= \bigcup_{k=1}^p S_k, \\ S_k \cap S_l &= \emptyset, \quad \forall k, l \in K, \\ \sum_{a_i \in S_k} d_i &\leq Q, \quad \forall k \in K. \end{aligned} \quad (1)$$

Fig. 1 shows a pictorial representation of a CCP solution with $p = 3$ and $n = 16$ whereas a CCP solution is represented as follows.

From complexity point of view, the class of clustering problems are among the most difficult combinatorial optimization problems. Brucker (1977) proved that the uncapacitated clustering problem (UCP) with $p = 2$ is $\mathcal{NP}$ -complete, from which the NP-completeness of the CCP can also be deduced. The CCP has a wide range of practical applications including: location of switching centers in communication networks (Mirzaian, 1985), construction of optimal index funds (Beck and Mulvey, 1982), consolidation of customer orders to vehicle shipment (Koskosidis and Powell, 1992), information system design (Karimi, 1986; Klein

and Aronson, 1991), manufacturing and marketing applications (Vakharia and Mahajan, 2000), location of offshore platforms (Hansen et al., 1994). For other applications, we refer to Mulvey and Beck (1984), Chhajed et al. (1993), Osman and Christofides (1994) and Hansen and Jaumard (1997). The CCP has been tackled with different metaheuristics and exact algorithms including:

- *Classical heuristics*: an iterative sub-gradient optimization heuristic by Mulvey and Beck (1984), and its extensions by Koskosidis and Powell (1992).
- *Metaheuristics*: Hybrid simulated annealing and tabu search by Osman and Christofides (1994); bionomic algorithm by Maniezzo et al. (1998); adaptive tabu search by Franca et al. (1999); guided construction search by Osman and Ahmadi (2002) and the density based problem space search by Ahmadi and Osman (2004).
- *Exact solutions based on*: a column generation approach by Hansen et al. (1993); and a set partitioning approach by Baldacci et al. (2002).

Metaheuristics are a class of approximate methods that have widespread success in tackling complex optimization problems (Osman, 1995; Osman and Laporte, 1996; Hansen and Ribeiro, 2002; Voss et al., 1998; Osman and Kelly, 1996). They include the Greedy Adaptive Random Search Procedure (GRASP). It has been successful in solving combinatorial optimization problems Pitsoulis and Resende (2001) and Festa and Resende (2001). However, GRASP is memory-less. It does not make use of information generated at previous iterations. On the other hand, Adaptive Memory Programming (AMP) is another metaheuristic that is designed to use information to guide the construction of new solutions, Glover (1997). In this paper, a merger of the Greedy Adaptive Random Search Procedure and the Adaptive Memory Programming metaheuristic into a new GRAMPS framework is proposed. The remaining part of the paper is organized as follows. In Section 2, an introduction to GRAMPS metaheuristic is presented. In Section 3 GRAMPS implementation to the CCP is discussed. In Section 4 extensive computational results are reported on a standard

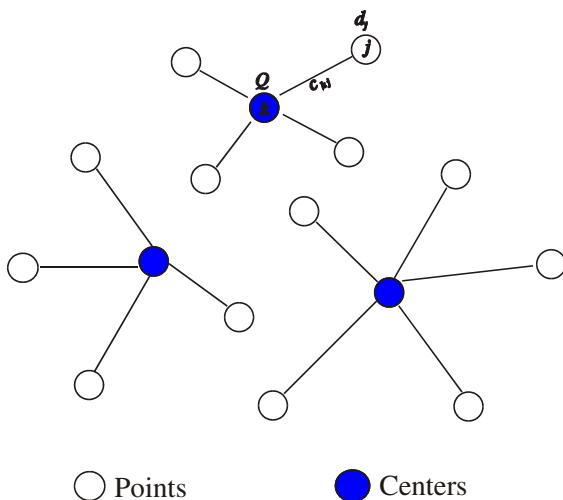


Fig. 1. A pictorial representation of a feasible solution with $p = 3$ and $n = 16$.

set of bench-marks and new sets of large instances, followed by a comparison with the best existing methods in the literature. In Section 5 we conclude with some remarks and directions on further research.

2. GRAMPS metaheuristic

Incorporating memory components into a memory-less procedure would make the resulting metaheuristic more intelligent and efficient in solving complex problems. However, there are questions related to memory: what are the memory components and their management, how to exploit memory information and guide the search process? In the literature, there are three classes of guided metaheuristics and their hybrids: guided construction search metaheuristic (GCSM) and a guided local search metaheuristic (GLSM) and guided population search metaheuristic (GPSM) (Osman, 1999, 2003). The class of GCSM includes GRASP, AMP, the ant colony optimization (ACO), and the problem space search (PS), among others. GCSM is superimposed on a good construction method to generate multi-start initial solutions for further improvements. The guided construction metaheuristics have similarities and differences which can have a big effect on their performance. In this section, we shall concentrate on the GCSMs that led to the derivation of the proposed Greedy Random Adaptive Memory Programming Search Procedure (GRAMPS).

GRASP is a multi-start iterative process. Each GRASP iteration consists of two phases: a construction phase and a local search phase, Feo and Resende (1989). In the first phase, a feasible solution is iteratively constructed by randomly choosing an element from a restricted candidate list (RCL) to enter the current partially constructed solution. RCL contains high quality elements which are generated according to a greedy adaptive function. The second phase is an improvement phase, where a local optimum in the neighborhood of the constructed feasible solution is sought. The best solution, found during a number of GRASP iterations, is the reported GRASP final solution. In GRASP, simple memory components are used to

store the best found solution. Another example of GCSM is the adaptive memory programming procedure. The origin of AMP goes back to the work on surrogate constraints by Glover (1977). In the literature, there are three types of AMP. The first type is a merger of components found in different good generated solutions. It was implemented by Rochat and Taillard (1995) and Golden et al. (1997) for vehicle routing problems. A pool of tours of elite solutions were progressively enriched. A complete solution was heuristically created from the pool by assembling tours with probabilistic preferences to those with lower costs. This AMP type is related to path relinking which makes use of AMP principles for joining solutions, Glover et al. (2000). The second type uses adaptive memory principles to store information on individual elements found in a pool of previously generated solutions. The individual information are adaptively added to the initial problem data to enable the construction of different initial solutions. Computational experiments for the quadratic assignment problem showed that this AMP type improves significantly over other multi-start methods which do not incorporate memory, Fleurent and Glover (1999). The third type is ACO of Dorigo et al. (1996). In ACO, information from previously constructed solutions are used in the construction of the next solutions using a probabilistic function. Similar to the second type, ACO combines greedy and probabilities measures. However, the main differences between AMP and ACO are in the collection of information, different probability functions, and the management of the pool of solutions. ACO gathers information from all solutions while the former collects information from a set of elite solutions. ACO also uses the concept of evaporation which does not exist in AMP. Glover (1997) stated that adaptive memory programming is still in its infancy and a few implementations exist. Hence, introducing GRAMPS will make a nice contribute to the development of knowledge on the guided construction search methods.

In general, GRAMPS belongs to the class of guided construction metaheuristic. Each iteration of GRAMPS consists of three processes: construction, learning and local search. The construction

process construct a feasible solution iteratively by choosing elements from a restricted candidate list (RCL) to enter the partially constructed solution, according to a biased probability function that favors top elements. RCL contains high quality elements which are generated according to a greedy adaptive function. The evaluation function combines problem-domain data with memory-domain data. The learning process collects information about individual features found in good solutions. The collected information are strategically combined with problem-domain data to guide the construction in the problem-domain space. At early phase of the construction process, priorities are given to problem-domain data and progressively shifted towards collected memory-domain data as the learning increases. After completing the construction of a feasible solution, it is then improved by an efficient local search procedure defined by the local search process. At the end of local search process, the learning process updates the collected information according to some rules governing the addition/removal of information from the memory bank. The processes are continued for a number iterations. The best solution found during the search is reported as the GRAMPS final solution.

It can be seen that GRAMPS is similar to GRASP in two processes. But it adds to it the biased probability selection and the learning process. Both ACO and GRAMPS share common concepts in having a learning process and a local search process. But they are different in several ways. GRAMPS learning is similar to the AMP type in Fleurent and Glover (1999). It collects information on special features of the elite solutions. Elite solutions are also admitted into the pool of solutions, provided they pass some tests on aspiration criterion, similarity and diversity measures. Another difference is in the management of the pool of elite solutions. In special cases, the ACO learning process may cause problems. For instance, if low quality solutions are found with high frequency in the search, then this would change the pattern of the probability selection and would allow the inclusion of weaker elements into the constructed solutions. The role of the evaporation concept in ACO is devised to reduce this

effect. However in GRAMPS, this effect is less likely to happen due to the different management of information.

3. GRAMPS implementation

In this section, GRAMPS implementation to the CCP is described.

3.1. The construction process

GRAMPS relies on a good constructive heuristic to generate multiple initial solutions. The GRAMPS construction process uses the density search construction method (DSCM) which was developed for the CCP in Ahmadi (1998). DSCM uses the concept of density of points to select centers and the regret concept to assign points to centers. It also employs elements of adaptive computation for both density and regret values, and a periodic construction–deconstruction phase after building every cluster.

Let us define the following, for each unassigned point x_i , let X_i be the set of its μ_i -nearest points such that

$$\sum_{a_k \in X_i} d_k \leq Q \quad \text{and} \quad \mu_i \leq \frac{n}{p}. \quad (2)$$

Let D_i be the density value of x_i defined as

$$D_i = \frac{\mu_i}{T_i}, \quad (3)$$

where T_i is the scatter of point x_i —the total sum of its distances to all points in X_i . The center ϕ_j of cluster S_j is then the corresponding point associated with the smallest T_i value for all points $x_i \in S_j$. Let R_i be the regret value of x_i defined as the difference between the distances from x_i to its first ϕ_{i_1} and second ϕ_{i_2} nearest centers.

The DSCM procedure consists of two phases. In the first construction phase and at iteration k , DSCM computes the density D_i values for all unassigned points. The D_i values are then sorted in decreasing order in a list L . The point x_i at the top of L is selected to be the temporary k th center. The points in X_i are then assigned to x_i to

create the k th cluster. After the assignment, the new permanent center, ϕ_k is recomputed. The search continues until all points are assigned to a center. In the second construction–deconstruction phase, given the set of centers obtained in the first phase, the points are then reassigned to their nearest center in decreasing order of their regret values. After the assignment is complete, the set of centers is recomputed and the process is repeated until no change in the set of centers. Further details can be found in Ahmadi and Osman (2004).

3.2. The learning process

Adaptive memory plays an integral part of our GRAMPS implementation. From the search history, information on good elements in a set of elite solutions are collected in a memory bank. A learning process is normally designed to exploit such information to guide the *constructive* process. It is based on the AMP notions of *strongly determined* and *consistent variables*. Given, a set E of r elite solutions, a *strongly determined* variable is the one whose value cannot be changed except by inducing a disruptive effect on the objective function value or on the values of the other variables. The *strength* of this variable is usually measured by the quality of the elite solutions in which it lies with its particular value. A *consistent variable* is one that is frequently strongly determined at a particular value. The measure of the consistency is usually defined as the *frequency* of being strongly determined at a particular value in the current set E of the elite of solutions. This information, combined with the other measures of attractiveness, is translated into a probabilistic evaluation of points.

In a CCP solution, the selection of centers is more strategic than assignment of points. With a bad set of centers, the construction–deconstruction phase has limited ability in removing the bad center. Hence, gathering information on the quality of centers can guide DSCM to construct high quality solutions of good centers. Therefore, the learning process will collect information from the pool E of elite solutions on centers in the solutions.

3.2.1. Management of elite set of solutions

The pool E is a matrix of dimensions $(n + p + 1) \times r$ where each column is associated with a given solution S and r is the number of elite solutions. In a given column, the first row stores the objective function value Z , the next p rows record the p centers and the last n rows indicate the assignment of points to centers for all a_i , $1 \leq i \leq n$. Initially, the pool E is empty. Let B and W be the best and worst solutions in E , respectively. After finding a new solution S , it is then compared with each of the elite solutions in E to decide whether S should be admitted into E or not according to certain similarity measure. Our similarity measure between solutions S and Y is based on the number of entries with the same value in the rows $p + 2$ to $n + p + 1$ of their corresponding columns in E . Let Y be the elite solution with the highest similarity to S and let η be their similarity value, then solution S is *admitted* into E if:

1. $|E| = 0$ $\{E$ is empty $\}$;
2. $|E| > 0$, $\eta < n\beta$ and $Z_S < Z_W$ $\{S$ is better than the worst solution, and different enough from the other elite solutions $\}$;
3. $|E| > 0$, $\eta \geq n\beta$ and $Z_S < Z_Y$. $\{This$ is an *aspiration* criterion. S has similarity above a threshold, but it has a better objective value than Y $\}$.

After admission of S into E , the following replacement criterion is used. If $|E| < r$, S is added to the last $|E| + 1$ position in E . Otherwise, it replaces the worst solution W or the corresponding similar solution Y in the aspiration case. Note, the elite set of solutions would not be updated until it is full, i.e., $|E| = r$.

3.2.2. Strongly determined and consistent centers

A point is *strongly determined* if it is a center in an elite solution S in E . The strength of a center is measured by the quality of its solution S and it is normalized as Z_B/Z_S where Z_B is the best objective value in E . The consistency of a center is measured by the number of times it appears as a center in the set of elite solutions. The strength and consistency of a center ϕ_i are combined in one *Intensity* measure defined as

$$I_i = \sum_{\varphi_i \in S, S \in E} \frac{Z_B}{Z_S}. \quad (4)$$

Note that, for every point $a_i \in A$, we have $0 \leq I_i \leq |E| \leq r$ i.e., a_i can appear zero time at which $I_i = 0$, or up to a maximum of $|E|$ times. The intensity measure contains information on the frequency of a_i appearance and on the quality of the elite solutions in which a_i is a center. The higher is the intensity value, the more desirable is to select a_i as a center.

3.2.3. The probabilistic selection criterion

The intensity values can be used within DSCM to design a greedy randomized function to enforce the selection of centers with high intensity values. A new evaluation function, θ_c , is designed based on a linear combination of the normalized density and intensity values. Let D_m be the highest density among all unassigned points, and let δ_i be the normalized density value of point a_i defined as

$$\delta_i = \frac{D_i}{D_m}.$$

The evaluation function θ_c is then defined as

$$\theta_c(a_i) = c\delta_i + I_i, \quad (5)$$

where c is a *balancing parameter*. Using (5), probabilities can be assigned to strongly bias the selection of points with high θ_c values as centers. Let RCL be a restricted candidate list of unassigned points whose θ_c evaluations are within $\alpha\%$ of θ_m , the highest θ_c value. Then, for every point $a_i \in \text{RCL}$, the probability of a_i is defined as

$$p_i = \frac{\theta_c(a_i)}{\sum_{a_j \in \text{RCL}} \theta_c(a_j)}.$$

An unassigned point can then be randomly selected as center as follows. Let ρ be a random value drawn from the uniform distribution on $U = [0, 1)$. The U interval is partitioned into sub-intervals $U_1, \dots, U_l$, where $U_i = [\sum_{j=1}^{i-1} p_j, \sum_{j=1}^i p_j]$ so that each point $a_i \in \text{RCL}$ would have one interval with a probability $p_i = \sum_{j=1}^i p_j - \sum_{j=1}^{i-1} p_j$. The interval, which contains the random value ρ , would identify the point to be a center.

A main question arises. What is the merit of combining the δ_i and I_i values and do they carry

the same information? The constructed solutions are multi-starts, each start is further improved by the local search process. Therefore, the information obtained from the intensity values will be different from the information generated from the density values. Hence, different information will be provided by both intensity and density values on high quality centers. A final point, restricting the greedy selection to only the intensity values with $c = 0$, will intensify the search too strongly and in most of cases only a few solutions will be generated. Note that, our evaluation function generalises that of Rochat and Taillard (1995) used for the vehicle routing problem as the latter can be obtained by setting $c = 0$.

3.2.4. Control of the balancing parameter

The balancing parameter c plays an important role in adjusting the weight given to the greedy value, δ_i , and the intensity value, I_i . Hence, different emphases to historical intensity and input density values will be given. Note that, $I_i \leq r$ for any point a_i , if $c = r$ then the same weight will be given to density and intensity values provided a sufficient number of elite solutions are found. Therefore, at an early stage of the search where less information are available, more weight should be given to δ_i . Consequently, c is initialized to a high value, and it is updated periodically based on the availability of the diverse elite solutions. In our implementation, the values of c can increase/decrease depending on the number of the distinct solutions produced in the last $2r$ iterations. If this number is lower than a pre-specified number r_1 , the search needs to be diversified to generate more different solutions by increasing the value of c and the value of α that determines the range of the selection from the RCL list as follows:

$$\begin{aligned} c &= \min\{c + r, 10r\} \quad \text{and} \\ \alpha &= \min\{\alpha + 0.1, 1\}. \end{aligned} \quad (6)$$

However, if the number of the distinct solutions is higher than a pre-specified number r_2 , the search needs to intensify by decreasing the values of c and α to

$$c = \max\{c - r, r\} \quad \text{and} \quad \alpha = 0.2. \quad (7)$$

3.3. The local search process

The local search process is designed to improve the quality of constructed solutions by the construction process. It employs a local search descent procedure, which starts from an initial solution S . It generates a set of neighboring solutions Ω that improves upon the objective value of S . It then selects a neighbor $S' \in \Omega$ to replace the current solution S according to certain criterion, such as first-improve (FI) or best improve (BI) strategies. The search continues until the set Ω is empty. The final solution S is declared to be a local optimum with respect to the neighborhood generation mechanism which defines the implemented operators applied on S to generate the set neighbors, $N(S)$.

A restricted 1-interchange generation mechanism is implemented for the local search descent procedure. It is based the concepts of neighborhood of clusters and points that are superimposed on the λ -interchange neighbourhood mechanism of Osman and Christofides (1994). Given a pair of clusters (S_k, S_l) in S , the pair is said to be μ -adjacent, if there exists a point $x_i \in S_k$ and a point $x_j \in S_l$ such that x_i is among the μ -nearest points of x_j and vice versa. The restricted 1-interchange neighborhood of S is denoted by $\overline{N}_1(S)$. It is generated by considering all pairs of μ -adjacent clusters and performs all possible 1-interchange feasible moves between them. A 1-interchange move is generated by a shift (or swap) operator. An operator interchanges λ_1 point from cluster S_k with λ_2 point from cluster S_l where $0 \leq \lambda_1, \lambda_2 \leq 1$ are integers. A shift operator sets λ_1 or λ_2 to zeros where in a swap operator sets both λ_1 and λ_2 to one. Note that $\overline{N}_1(S)$ is a subset of $N_1(S)$ generated by considering all $\frac{p(p-1)}{2}$ pairs of clusters. The restricted 1-interchange local search descent procedure is implemented with special data structures for efficient computation of centers and moves. The best-improve strategy is used to select among improving moves which are generated by restricting moves to 4-adjacent clusters. For further details, we refer to Ahmadi (1998) and Osman and Ahmadi (2002).

3.4. GRAMPS algorithm

The GRAMPS construction process attempts to randomize DSCM by using the evaluation

function θ_c instead of the density function D . Within DSCM, point a_i is selected as a center if it has the largest density value $D_m = \text{Max}\{D_j | a_j \in X\}$ among all unassigned points in X . This greedy selection criterion can be randomized to select one point from a restricted candidate list RCL of high evaluation centers. Let θ_m be the highest value $\theta_c(a_i)$ among all $a_i \in X$ and let α be a parameter in the range $[0, 1]$ to specify the percentage deviation from θ_m , then RCL is set of points defined as $\text{RCL} = \{a_j \in X | \theta_c(a_j) \geq (1 - \alpha)\theta_m\}$. The adaptivity of the computation is an intrinsic part of the DSCM due to the update of the evaluation values after each selection of a center. Hence, the main procedural difference between DSCM and GRAMPS construction is the selection of centers from from RCL instead of the point at the top of RCL. The multi-start GRAMPS construction procedure will be called the Randomized Density Search Constructive Method (RDSCM). The RDSCM solutions are improved by applying the restricted 1-interchange descent method as follows:

Definitions

Φ	set of centers
X	set of unassigned points in A
Y	set of currently assigned points
Θ	the evaluation function vector of θ_c values
θ_m	the highest θ_c values in RCL
α	the percentage deviation from θ_m to define RCL
<i>Max_Iter</i>	maximum GRAMPS iterations
<i>Major</i>	counter number for GRAMPS iterations
Z_B	the best objective value so far

Main RDSCM:

1. Set $k = k + 1$; {index of clusters}.
2. Set the $\theta_c(x_i)$ value to zero for each $x_i \in X$.
3. For every $x_i \in X$, compute its $\theta_c(x_i)$ value;
4. Sort the unassigned points in decreasing order of their $\theta_c(x_i)$ values;
5. Define the RCL list
6. Select point x_i from RCL to be the k th center, φ_k , according to probability p_i , and set: $\Phi = \Phi \cup \{\varphi_k\}$;
7. Update the sets X and Y ;
8. If $\{k \geq 2\}$ Then recompute the new set of centers and new assignments;

9. If $\{k < p\}$ Then go to Step (1);
10. Recompute the new set of centers and new assignments.

End of Procedure.

Main GRAMPS:

$Z_B = \infty$

While (Major < Max_Iter) do: {Major is a gramps iteration counter}.

1. Call RDSCM to get a solution S ;
2. If $\{S$ is a non-repeated initial solution}
Then
Call the restricted 1-interchange local search procedure;
Else
Go to Step 1.
End if
3. If $(Z_S < Z_B)$ then
- $Z_B = Z_S$;
-Save the current solution as the best solution so far;
-Major = Major +1;
End if

End While;

End GRAMPS.

4. Computational experience

4.1. Test instances

A series of computational experiments are conducted using set of test instances in the literature. The overall aim of our experiments is to show the usability of the memory structures of GRAMPS in producing an effective search algorithm and to show its competitiveness with other existing algorithms in the CCP literature. Table 1

contains a short summary about the characteristics of data instances generated by different authors.

The proposed metaheuristics are coded in Fortran 77 and run on a Sun SPARC Server 1000 with 50 MHz processor under the solaris 2.3 operating system. The quality of the solutions are reported in terms of the relative percentage deviation (RPD) of the heuristic objective value (Z_H) from the optimal Z_O , lower bound Z_{LB} or best objective Z_B value, whichever is available. The relative percentage improvement (RPI) in the objective value by a heuristic from an initial starting solution Z_I is also reported. These measures are computed as follows:

$$RPD = 100 \times \left(\frac{Z_H - Z_O}{Z_O} \right) \quad \text{and}$$

$$RPI = 100 \times \left(\frac{Z_I - Z_H}{Z_I - Z_O} \right).$$

Most of the results are expressed in terms of ARPD, the average of the RPD values and ARPI, the average of RPI values due to a large number of runs. Note that, the average CPU time to the best solution is also reported under the ATTB legend. We have conducted two set of experiments: the first experiments are run for $2n$ seconds to study the effect of GRAMPS components and identify their best setting values. The second experiments are run for $30n$ seconds with the best setting of parameters to compare the results with the other algorithms from the literature.

Finally, a new performance criterion is used to measure the Marginal relative Improvement per unit of CPU time (MIC) which is computed as the ratio of the ARPI value over the ACPU value. The ARPI values are computed from the DSCM

Table 1
Specifications of the set of 40 instances

Set no.	Instance indices	n	p	Tightness $\tau = \frac{\sum d_i}{pQ}$	Data (x, y)	Distributions, d_i	Reference
1	1–10	50	10	[0.63–0.83]	Normal	Uniform	Hansen et al. (1993)
2	11–20	50	5	[0.82–0.96]	Uniform	Uniform	Osman and Christofides (1994)
3	21–30	100	10	[0.85–0.94]	Uniform	Uniform	Osman and Christofides (1994)
4-i	31–35	150	15	0.83	Normal	Uniform	Ahmadi and Osman (2004)
4-ii	36–40	150	15	0.83	Uniform	Uniform	Ahmadi and Osman (2004)

solutions. The MIC measure derives a ranking of algorithms based on the quality of solutions produced and the computational effort used by each algorithm. It may be considered as an alternative comparative approach, rather than comparing solution quality and computation effort separately. For more details we refer to Osman (2003).

4.2. Effect of the learning process

To examine the effect of learning process on GRAMPS performance, two GRAMPS variants are investigated: V_1 is GRAMPS *without* the learning process and the use of the biased probability based on the memory information. In this variant, centers are then uniformly selected from a candidate list RCL consisting of the $\alpha\%$ of centers with the highest density values, similar to GRASP. The second variant, V_2 , is GRAMPS *with* the adaptive learning process and the use of biased probability function added to GRASP. Both variants are investigated on all 40 test instances using one single setting of parameters: the population size, $r = 10$; the percentage deviation from the best evaluation to determine the candidate list RCL, $\alpha = 20\%$; the similarity threshold value, $\beta = 0.8$; the lower limit on distinct solutions $r_1 = 0.2r$; and the upper limit on distinct solutions, $r_2 = 0.8r$. The reason for setting a relatively small value for r lies in the special structure of the CCP and RDSCM which produces solutions with high similarities. After a number of iterations, most of the new elite solutions are accepted through the aspiration criterion. Moreover, using a large value for r , a large number of the elite solutions will be identical, and a diverse set of r elite solutions may not be found. The value of $\alpha = 20\%$ has been

adopted after an extensive computational experiment with different values in Ahmadi (1998). It is found that larger values of α would generate very diverse solutions while smaller values would generate very similar solutions. Hence, an appropriate value for α must be chosen to provide a good balance between the two extremes. The density and intensity balancing parameter c is initialized with a large value of $c = 10r$. This choice would assign higher probabilities to the points with the high density values at the early stages of the construction search. The strategic update of c in Eqs. (6) and (7) would shift the bias towards intensity information at the later stages of the search.

To evaluate the performance of multi-start approaches, five measures were reported:

ARPD: average relative percentage deviation of best solutions.

ATTB: average total time to best solutions.

AITB: average iterations number to best solutions.

ANDS: average number of distinct solutions.

ARPD: average relative percentage deviation of all solutions.

V_1 : GRAMPS without the learning process and the probabilistic bias (GRASP).

V_2 : GRAMPS with the learning process and the probabilistic bias.

Table 2 presents the averages of the performance measures for the GRAMPS V_1 and V_2 variants using a maximum of $2n$ CPU time in seconds each. The effect of the learning process can be seen from the improved performance of V_2 over V_1 in terms of the average of ARPD of best solutions (ARPD) and the average of ARPD of all

Table 2
The results of the two GRAMPS variants for the short run of $2n$ seconds

Instances	GRAMPS (V_1)					GRAMPS (V_2)				
	ARPD	ATTB	AITB	ANDS	ARPD	ARPD	ATTB	AITB	ANDS	ARPD
1–10	0.09	2.04	19.4	32.7	4.59	0.09	2.17	13.3	20.4	4.77
11–20	0.00	0.82	7.2	4.0	3.40	0.00	0.52	4.9	2.4	2.61
21–30	0.10	26.93	44.8	32.1	3.60	0.09	43.99	64.7	38.3	4.53
31–40	2.66	97.18	47.6	55.5	7.41	2.60	111.16	56.7	54.3	6.99
Averages	0.71	31.74	29.75	31.08	4.75	0.69	39.46	34.90	28.85	4.72

generated solutions (ARPD). The relatively small amounts of improvements of ARPD in V_2 is due to the high quality of the DSCM initial solutions, which makes the percentage of the possible improvement very small in a short CPU time. This point can be illustrated from Tables 3–5 where DSCM produces 7 optimal solutions for the 30 instances from the literature and high quality solutions for all the others. In Table 2, V_2 found the best solutions for small-sized instances faster and in a smaller average of iteration numbers than V_1 . For the large sized instances, the iteration numbers to the best solutions by V_2 are bigger than that of V_1 due to the fact that better solutions are found due to the learning process but it is still within the same CPU time limit.

The effect of longer runs on the performance of both GRAMPS variants is also investigated. Each variant is allowed to run for a maximum CPU time of $30n$ seconds per instance. The average performance measures are depicted in Figs. 2 and 3 for V_1 and V_2 , respectively. From the figures, it can be demonstrated that extra CPU time improves the average quality of best solutions (ARPD). In particular, from Fig. 3, it is interesting to observe that the V_2 best solutions are found in a faster average of CPU time to best (94.44 seconds) with a larger average of number of iterations (130.95) than (136.19) seconds with (121.45) iterations for V_1 . The reason is that V_1 without learning starts from poorer constructed solutions as seen from the ARPD values and it may take longer CPU time

Table 3
Results for data sets 1 and 4 for single and multi-start methods

Sets $n \times p$	Single-start		Multi-start				
	DSCM	DSCM + BIDS	MB	GRAMPS	TTB	ITB	OPT
50×10							
1	11 871.8	11 620.9	11 620.9	11 620.9	0.31	1	11 620.9
2	13 229.3	13 229.3	13 403.3	13 147.9	1.32	8	13 147.9
3	13 275.7	13 275.7	13 371.4	13 275.7	1.21	8	13 275.7
4	12 154.4	12 154.4	11 778.3	11 778.3	1.86	12	11 778.3
5	11 317.7	11 303.7	11 586.5	11 289.6	0.75	4	11 289.6
6	13 407.5	12 459.9	12 391.1	12 391.1	2.52	20	12 391.1
7	12 815.8	12 815.8	12 815.8	12 815.8	1.56	10	12 815.8
8	13 364.2	12 740.3	12 498.1	12 387.7	352.49	2883	12 387.7
9	11 508.9	11 490.7	11 020.7	10 980.6	8.14	61	10 980.6
10	12 431.2	12 431.2	11 954.2	11 919.9	0.57	3	11 919.9
Averages	3.141 ^a	1.627 ^a	0.684 ^a	0.00^a	37.07 ^b	30 ^b	
150×15							LB
31	404.49	402.09	407.272	400.41^c	305.40	161	386.80
32	425.54	418.83	410.23	407.04^c	474.89	250	394.34
33	396.56	390.53	384.069	382.84	162.62	95	369.73
34	387.57	382.06	387.364	382.06^c	5.85	2	373.75
35	415.46	415.10	406.828	405.68^c	21.96	11	396.96
36	1319.67	1317.30	1292.19	1285.53^c	301.51	143	1247.04
37	1332.42	1328.78	1298.96	1290.08^c	98.16	46	1272.55
38	1283.01	1278.23	1254.5	1244.44^c	596.78	317	1234.69
39	1241.83	1237.96	1237.61	1230.00^c	62.84	29	1215.25
40	1400.06	1393.93	1375.36	1357.95^c	949.59	478	1307.20
Averages	5.190 ^a	2.297 ^a	0.845 ^a	0.005^a	297.96 ^b	153.20 ^b	

^a Average of ARPD over all instances.

^b Average of time and iterations to best.

^c Best found solutions.

Table 4
Results for data sets 2 and 3 for single and multi-start methods

Sets $n \times p$	Single-start		Multi-start				Literature			
	DSCM	DSCM + BIDS	Implemented				OC ^a	BA ^b	ATS ^a	OPT
			MB	GRAMPS	TTB	ITB				
50 × 5										
11	713	713	744	713	0.22	2	713	713	713	713
12	740	740	740	740	0.17	1	740	740	740	740
13	758	753	753	751	1.43	14	751	751	751	751
14	651	651	652	651	0.15	1	651	651	651	651
15	666	666	680	664	0.92	11	664	664	664	664
16	783	778	778	778	0.14	1	778	778	778	778
17	787	787	791	787	0.85	6	787	787	787	787
18	872	839	830	820	0.21	1	820	820	820	820
19	724	724	715	715	0.24	2	715	715	715	715
20	837	837	851	829	0.87	10	829	829	829	829
Averages	1.044 ^c	0.511 ^c	1.160 ^c	0.00 ^c	0.83 ^d	4.9 ^d	0.00 ^c	0.00 ^c	0.00 ^c	
100 × 10										OPT
21	1006	1006	1008	1006	4.63	7	1006	1006	1006	1006
22	974	970	966	966	27.59	45	966	966	966	966
23	1065	1056	1026	1026	8.80	10	1026	1026	1026	1026
24	1009	1009	993	982	37.56	63	985	982	982	982
25	1100	1099	1107	1092	20.47	32	1091	1091.8	1091	1091
26	983	979	962	954	25.56	44	954	954.2	954	954
27	1124	1123	1060	1034	32.67	56	1039	1034	1034	1034
28	1073	1062	1048	1043	8.33	14	1045	1043	1043	1043
29	1066	1055	1041	1032	187.33	284	1031	1031.4	1032	1031
30	1053	1051	1026	1012	69.13	92	1005	1013	1005	1005
Averages	3.099 ^c	2.678 ^c	0.968 ^c	0.09 ^c	42.21 ^d	64.70 ^d	0.1 ^c	0.09 ^c	0.01 ^c	

^a Best solutions in a single run.

^b Average of best solutions over 5 runs.

^c Average of ARPD over all instances.

^d Average of time and iterations to best.

Table 5
The average CPU time over all instances by the compared algorithms

Sets	Single-start		Multi-start				
	DSCM	DSCM + BIDS	MB	GRAMPS	OC	BA	ATS
11–20	0.340	0.515	0.288	0.520	2.410	0.090	41.440
21–30	0.790	1.593	614.448	42.206	70.710	161.490	500.460
ACPU	0.565 ^a	1.054 ^a	307.5 ^a	21.36 ^a	179.2 ^b	80.79 ^c	270.95 ^d
ACPU ^c	0.565	1.054	307.5	21.36	8.615	145.422	785.755
ARPD	2.071	1.594	1.604	0.045	0.050	0.045	0.005
MIC	–	23.032	0.158	4.580	11.327	0.672	0.127
Ranks	–	–	4	2	1	3	5

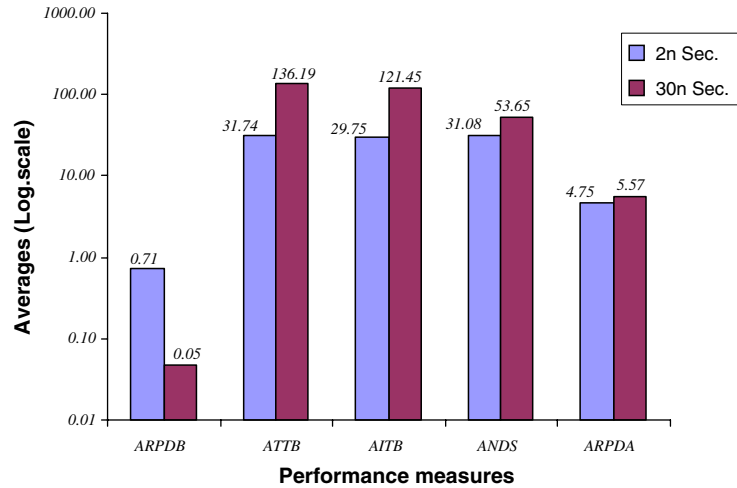
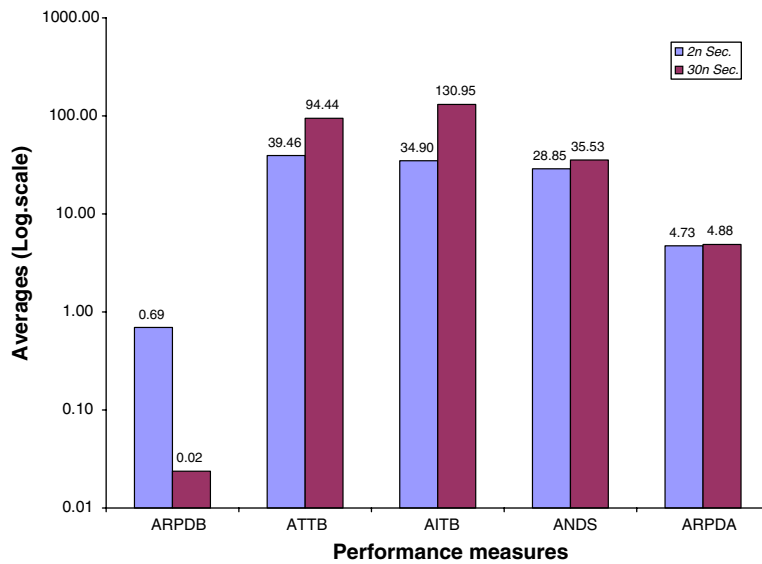
^a CPU time on a Sun SPARC 1000 server (10 Mflop/s).

^b CPU time on a VAX 8600 (0.48 Mflop/s).

^c CPU time on an IBM PC Pentium 166 MHz (18 Mflop/s).

^d CPU time on a Sun SPARC 20 (29 Mflop/s).

^e ACPU time converted into an equivalent time on Sun SPARC server 1000.

Fig. 2. Performance measures for GRAMPS (V_1) without the learning process.Fig. 3. Performance measures for GRAMPS (V_2) with the learning process.

to improve. The overall results demonstrate that using learning memory produces more effective algorithm than memory-less counterpart. Therefore, GRAMPS variant V_2 is recommended for a comparison with other algorithms in the literature and it will be referred simply as GRAMPS.

4.3. Comparison with the existing algorithms

From the literature, the following heuristics and metaheuristics are used for the comparison: the hybrid simulated annealing and tabu search (OC), the binomic algorithm (BA), the adaptive tabu

search (ATS), and the density search construction method (DSCM). The metaheuristics' results for OC, BA and ATS are taken from Osman and Christofides (1994), Maniezzo et al. (1998) and Franca et al. (1999), respectively. The results for the DSCM construction heuristic are taken from Ahmadi and Osman (2004). However, the DSCM is again implemented with the restricted 1-interchange neighbourhood and the best-improve selection strategy (DSCM + BIDS). GRAMPS are run for a maximum of $30n$ seconds per instance. The MB construction heuristic of Mulvey and Beck (1984) was allowed to run for an equal amount of $30n$ CPU seconds. The optimal solutions for data set 1 (1–10) are obtained with a branch and bound with column generation in Hansen et al. (1993) and that for data sets (11–20 and 21–30) are from Maniezzo et al. (1998) and Baldacci et al. (2002). For the large-sized data sets 4-i and 4-ii (31–40) their linear programming (LB) their best solutions are reported in Ahmadi and Osman (2004) and Ahmadi (1998). For each algorithm, the best obtained solution per instance is reported and the corresponding ARPD and the average CPU time to the best as reported by the authors. Moreover, for GRAMPS, more details on the total time to and the best solution (TTB) and the corresponding number of iterations to best (ITB) are reported to guide future research.

From Tables 3 and 4, a number of remarks can be made. First, the DSCM + BIDS implementation shows how much improvement can be made if it was not embed within GRAMPS. DSCM and DSCM + BIDS produce single solutions per instance and they are able to find the optimal solutions for 7 and 9 instances from literature (respectively) in very short CPU time. DSCM + BIDS continuously improves the quality of DSCM with a slight increase in the cpu time. On the contrary, MB does not always improve over DSCM + BIDS despite the $30n$ cpu time seconds allowed. This remark is observed from comparisons of the averages on data set 2 (11–20). Due to the consistency of DSCM-BIDS, and the existence of good construction and local search processes, it is used for a GRAMPS implementation.

Second, GRAMPS is able to find all the optimal solutions for instances in data sets 1 (1–10) and 2

(11–20) in a reasonable CPU time. However, the cpu time for instance 8 in data set 1 was out of order probably due to its tightness value of 0.66—the ratio of total weights of point to the total capacity of clusters. This light tightness may increase the time of the search as many unsuccessful feasible moves can be attempted. Last, for data set 4 (31–40), GRAMPS finds all the best known solutions, except for instance 33 for which a slightly better value of (382.67) was found during various experimentations by Osman and Ahmadi (2002). The GRAMPS solution for this instance is 382.84 which is very close to the best known solution. Note that GRAMPS results are reported using one specific setting of parameters described in the text to give a general guideline for other implementations. Further tuning of parameters for a given instance may lead to better results, but this kind of tuning is not helpful for solving other new instances.

Regarding comparison with other algorithms from the literature, data sets 2 (11–20) and 3 (21–30) are used by many authors in the literature. The corresponding results by various algorithms are reported in Table 4. All algorithms found the optimal solutions for data set 1. However, for data set 2 (11–30) different results are obtained. The optimal solutions found by each algorithm are highlighted in bold. Comparing the algorithms based on the average quality of solutions, they can be ranked from first to fourth in the following order: ATS, GRAMPS, BA and OC. However, considering the number of best know solutions found as a comparative measure, the ranking order would change to ATS, OC, GRAMPS, and BA.

Comparing the algorithms based their computational requirements is not an easy task. The speed of an algorithm does not depend on the CPU of the machine used, but also on cache, memory, compilers, and others. In Table 5, we report the performance of different machines in terms of Mflop/s (millions of floating-point operations per second) for the data sets in Table 4. The bench-mark results are taken from Dongarra (2001) for Mflop/s of different machine. The Mflop speed factor will be used to derive the adjusted CPU time of each algorithm in Table 5.

Comparing the algorithm based on the adjusted CPU time, the ranking order from most efficient to the least would become: OC, GRAMPS, BA and ATS. As it can be seen that it is difficult very difficult to compare based on a single measure, another measure (marginal improvement per a CPU unit, MIC) is used. It takes into the account both the quality of the solution and the CPU time of an algorithm to derive a ranking order of the compared the algorithms. The average relative percentage improvements from the DSCM values are computed for the derivation of MIC values. With this measure the ranking order was: OC, GRAMPS, BA and ATS. The MIC ranking should be used only as a rough guideline for evaluation of the algorithms as it only uses one of the factors responsible for the speed, namely the CPU-time. From the average of the three different measures it can be seen that GRAMPS is ranked second after the OC metaheuristic and better than the other two published metaheuristics. Hence, it makes a nice alternative approach to solving real-life CCP instances.

5. Conclusion

The paper describes the results obtained by applying a guided construction search metaheuristic for the capacitated clustering (p -median) problem, called greedy random adaptive memory programming search (GRAMPS) algorithm. GRAMPS is superimposed on a density construction search method (DSCM) and the restricted 1-interchange descent procedure (BIDS). The sub-heuristics are guided using a learning process that gathers historical information to be combined strategically with data input. The overall results showed that the GRAMPS compares very well with the best existing algorithms in the literature. The proposed merger of memory concepts and probabilistic measures has a good potential in designing effective multi-start approaches. Further research investigation in this direction should be encouraged.

The GRAMPS implementation can be improved in different ways. Apart from the fine-tunings of parameters, more sophisticated learning

process can be used to analyze information on other attributes in elite solutions, such as assignments of points to the centers. Another approach is to maintain a large set of elite solutions generated by different metaheuristics to initialize the learning process. This approach may provide a unified framework to combine the most powerful components of each metaheuristics to design a customised combination of them for specific instances. Another direction can be followed by using GRAMPS with path-relinking approach to explore trajectories that connect high quality solutions similar to Laguna and Marti (1999) and Aiex et al. (2003).

Acknowledgements

This research was supported by grants from University of Alzahra in Iran and the American University of Beirut in Lebanon. The authors would like to thank our sponsors for their kind supports. The authors are also grateful to the referees and the guest editors for their useful comments and suggestions.

References

- Ahmadi, S., 1998. Metaheuristics for the capacitated clustering problem. PhD thesis, University of Kent at Canterbury, UK.
- Ahmadi, S., Osman, I.H., 2004. Density based problem space search for the capacitated clustering problem. In: Derigs, U., Voss S. (Eds.), *Annals of Operations Research on Metaheuristics* (special issue), forthcoming.
- Aiex, R.M., Resende, M.G.C., Pardalos, P.M., Toraldo, G., 2003. GRASP with path relinking for the three-index assignment problem. *INFORMS Journal on Computing*, forthcoming.
- Baldacci, R., Hadjiconstantinou, E., Maniezzo, V., Mingozzi, A., 2002. A new method for solving capacitated location problems based on a set partitioning approach. *Computers & Operations Research* 29 (4), 365–386.
- Beck, M.P., Mulvey, J.M., 1982. Constructing optimal index funds. Technical report, Princeton University Report No. EES-82-1.
- Brückner, P., 1977. On the complexity of clustering problems. In: Henn, R., Korte, B., Oettli, W. (Eds.), *Optimization and Operations Research: Proceedings of a Workshop Held at the University of Bonn, 2–8 October*. Springer Verlag, pp. 45–54.

- Chhajed, D., Francis, R.L., Lowe, T.J., 1993. Contributions of operations research to location analysis. *Location Science* 1, 263–287.
- Dongarra, J.J., 2001. Performance of various computers using standard linear equations software. Technical report, Computer Science Dept., University of Tennessee, Knoxville. Available from <<http://www.netlib.org/benchmark/performance.ps>>.
- Dorigo, M., Maniezzo, V., Coloni, A., 1996. The ant system: Optimization by a colony of cooperating agents. *IEEE Transactions on Systems, Man, and Cybernetics-Part B* 26 (1), 29–41.
- Feo, T.A., Resende, M.G.C., 1989. A probabilistic heuristic for a computationally difficult set covering problem. *Operations Research Letters* 8, 67–71.
- Festa, P., Resende, M.G.C., 2001. Grasp: An annotated bibliography. In: Hansen, P., Ribeiro, C.C. (Eds.), *Essays and Surveys on Metaheuristics*. Kluwer Academic Publishers.
- Fleurent, C., Glover, F., 1999. Improved constructive multistart strategies for the quadratic assignment problem using adaptive memory. *INFORMS Journal on Computing* 11, 198–204.
- Franca, P.M., Sosa, N.G., Pureza, V.M., 1999. An adaptive tabu search approach for solving the capacitated clustering problem. *International Transactions in Operational Research* 6, 665–678.
- Glover, F., 1977. Heuristics for integer programming using surrogate constraints. *Decision Sciences* 8 (1), 156–166.
- Glover, F., 1997. Tabu search and adaptive memory programming—advances, applications and challenges. In: Barr, R.S., Helgason, R.V., Kennington, J.L. (Eds.), *Advances in Metaheuristics, Optimization, and Stochastic Modeling Technologies*. Kluwer, Boston, MA, pp. 1–75.
- Glover, F., Laguna, M., Marti, R., 2000. Fundamentals of scatter search and path relinking. *Control and Cybernetics* 29, 653–684.
- Golden, B.L., Laporte, G., Taillard, E.D., 1997. An adaptive memory heuristic for a class of vehicle routing problems with minmax objective. *Computers & Operations Research* 24, 445–452.
- Hansen, P., Jaumard, B., 1997. Cluster analysis and mathematical programming. *Mathematical Programming* 79, 191–215.
- Hansen, P., Ribeiro, C.C., 2002. *Essays and Surveys on Metaheuristics*. Kluwer Academic Publishers.
- Hansen, P., Jaumard, B., Sanlaville, E., 1993. Weight constrained minimum sum-of-stars clustering. Technical report, Gerad Technical Report G-93-38.
- Hansen, P., Pedrosa, E.D., Ribeiro, C.C., 1994. Modeling location and sizing of offshore platforms. *European Journal of Operational Research* 72, 602–606.
- Karimi, J., 1986. An automated software design methodology using capo. *Journal of Management Information Systems* 3, 71–100.
- Klein, K., Aronson, J.E., 1991. Optimal clustering: A model and method. *Naval Research Logistics* 38, 447–461.
- Koskosidis, Y.A., Powell, W.B., 1992. Clustering algorithms for consolidation of customer orders into vehicle shipments. *Transportation Research* 26B, 365–379.
- Laguna, M., Marti, R., 1999. GRASP and path relinking for 2-layer straight line crossing minimization. *INFORMS Journal on Computing* 11 (1), 44–52.
- Maniezzo, V., Mingozzi, A., Baldacci, R., 1998. A bionomic approach to the capacitated p -median problem. *Journal of Heuristics* 4, 263–280.
- Mirzaian, A., 1985. Lagrangian relaxation for the start–star concentrator location problem: Approximation algorithms and bounds. *Networks* 15, 1–20.
- Mulvey, J.M., Beck, M.P., 1984. Solving capacitated clustering problems. *European Journal of Operational Research* 18, 339–348.
- Osman, I.H., 1995. An introduction to metaheuristics. In: Lawrence, M., Wilsdon, C. (Eds.), *Operational Research Tutorial Papers*. Operational Research Society Press, Birmingham, pp. 92–122.
- Osman, I.H., 1999. A Unified-Metaheuristic Framework. *Lecture Notes in Artificial Intelligence*, vol. 1611, pp. 11–12.
- Osman, I.H., 2003. Meta-heuristics: Models, analysis, and directions. In: A tutorial paper presented at the joint EURO/INFORMS meeting, Istanbul, 6–10 July.
- Osman, I.H., Ahmadi, S., 2002. Guided construction search for the capacitated p -median problem. Working paper, School of Business, American University of Beirut, Lebanon.
- Osman, I.H., Christofides, N., 1994. Capacitated clustering problems by hybrid simulated annealing and tabu search. *International Transactions in Operational Research* 1, 317–336.
- Osman, I.H., Kelly, J.P., 1996. Metaheuristics: An overview. In: Osman, I.H., Kelly, J.P. (Eds.), *Metaheuristics, Theory and Applications*. Kluwer Academic Publishers, Boston.
- Osman, I.H., Laporte, G., 1996. Metaheuristics: A bibliography. *Annals of Operations Research* 63, 513–623.
- Pitsoulis, L.S., Resende, M.G.C., 2001. Greedy randomized adaptive search procedures. In: Pardalos, P.M., Resende, M.G.C. (Eds.), *Handbook of Applied Optimization*. Oxford University Press.
- Rochat, Y., Taillard, É.D., 1995. Probabilistic diversification and intensification in local search for vehicle routing. *Journal of Heuristics* 1, 147–167.
- Vakharia, A.J., Mahajan, J., 2000. Clustering of objects and attributes for manufacturing and marketing applications. *European Journal of Operational Research* 123, 640–651.
- Voss, S., Martello, S., Osman, I.H., Roucairol, C., 1998. *Meta-Heuristics: Advances and Trends in Local Search Paradigms for Optimization*. Kluwer Academic Publishers.